

Epitech Technology

MSc2

Travel Order Resolver

Extraction d'itinéraires de train à partir de requêtes en langage naturel

Rapport d'expérimentation

Équipe de projet

- Adrien GARNIER
- Chi NGUYEN
- Nathan HAN
- Samir MIGUEL
- Zeyu HOU

Date : 4 mars 2026

Année académique 2025–2026

Table des matières

1	Introduction	1
1.1	Contexte	1
1.2	Objectifs du projet	1
2	Modèles de traitement du langage	2
2.1	Baseline à base de règles	2
2.2	Présentation de spaCy	2
2.2.1	Modèles sm, md, lg	2
2.2.2	Modèle spaCy <code>blank</code> (coquille vide)	3
2.3	Présentation de CamemBERT	3
2.3.1	Rappels sur BERT et les Transformers	3
2.3.2	CamemBERT pour la NER en français	3
3	Protocole expérimental	4
3.1	Dataset et splits (1000, 3000, 5000, full)	4
3.2	Paramètres d'entraînement (itérations, epochs, loss)	4
3.2.1	Entraînement spaCy	4
3.2.2	Entraînement CamemBERT	5
3.3	Post-traitement des entités (fonction <code>_clean_entity</code>)	5
4	Résultats et analyses	6
4.1	Comparaison baseline / spaCy / CamemBERT	6
4.2	Impact de la taille du dataset	6
4.2.1	Comparaison des variantes spaCy	6
4.2.2	Comparaison des variantes CamemBERT	7
4.3	Sous- ou sur-apprentissage, courbes de loss	7
4.4	Discussion : 1000 exemples comme compromis suffisant	7
5	Module de pathfinding	8
5.1	Modélisation du réseau de gares	8
5.2	Algorithme de Dijkstra avec NetworkX	8
5.3	Mesure de la durée et routes optimales	8
5.4	Intégration dans le pipeline complet	9
6	MVP0 et déploiement	10
6.1	Choix du modèle final (spaCy blank 1k vs CamemBERT 1k)	10
6.2	Considérations de performance et de ressources	10
6.3	Interface utilisateur minimale (champ de texte)	10
6.4	Perspectives d'évolution	11

7	Conclusion	12
7.1	Bilan des expérimentations	12
7.2	Améliorations possibles	12

Chapitre 1

Introduction

1.1 Contexte

L’objectif du projet *Travel Order Resolver* est de transformer une demande de voyage en français, formulée en langage naturel, en un itinéraire de train optimal au sein du réseau ferroviaire français. Par exemple, à partir de la phrase « Je voudrais aller de Paris à Lyon », le système doit extraire automatiquement l’origine et la destination, puis calculer l’itinéraire le plus rapide entre ces deux gares.

Le système s’appuie sur deux briques principales :

- un module de NLP pour l’extraction d’entités nommées (Named Entity Recognition, NER) d’origine et de destination ;
- un module de pathfinding basé sur l’algorithme de Dijkstra et la librairie NetworkX pour trouver le plus court chemin en durée sur un graphe de gares.

Ce rapport présente le contexte, les choix de modèles (baseline, spaCy, CamemBERT), les expérimentations menées (notamment sur un sous-ensemble de 1000 phrases), les résultats quantitatifs, ainsi que les justifications des choix techniques au regard du cas d’usage et des contraintes de ressources.

1.2 Objectifs du projet

Le projet vise à concevoir et évaluer une chaîne complète allant de la requête en langage naturel jusqu’à la proposition d’itinéraire optimal, en étudiant différents modèles de NER et en mesurant les compromis entre précision, temps d’inférence et consommation de ressources.

Chapitre 2

Modèles de traitement du langage

2.1 Baseline à base de règles

La baseline est un modèle « fait maison » basé sur :

- des règles linguistiques simples (détection de prépositions de type « de », « à », « vers ») ;
- un gazetteer de 541 villes/gares SNCF ;
- des heuristiques pour décider quels tokens avant/après la préposition correspondent à l'origine ou à la destination.

Cette baseline atteint :

- **Accuracy** : 67,13 % ;
- **F1 Score global** : 88,41 % ;
- temps d'inférence autour de 17,38 ms par phrase.

Ce modèle montre rapidement ses limites :

- forte dépendance aux formulations exactes ;
- difficulté avec les noms composés (« La Ferté-Imbault »), la ponctuation et les variations de syntaxe ;
- difficile à maintenir et à généraliser quand le vocabulaire ou les patrons de phrases évoluent.

2.2 Présentation de spaCy

spaCy est une bibliothèque de NLP en Python orientée production, qui fournit des pipelines pré-entraînés pour différentes langues : tokenisation, POS tagging, parsing, NER, etc. Les pipelines français typiques sont :

- **fr_core_news_sm** (*sm*, small) : modèle léger, rapide, avec peu de vecteurs de mots ;
- **fr_core_news_md** (*md*, medium) : taille moyenne, inclut des vecteurs de mots pour une meilleure représentation sémantique ;
- **fr_core_news_lg** (*lg*, large) : modèle plus gros avec plus de vecteurs et de paramètres, plus précis mais plus coûteux en mémoire et en temps.

2.2.1 Modèles *sm*, *md*, *lg*

Les modèles *sm*, *md* et *lg* sont des pipelines généralistes qui couvrent un ensemble large de tâches NLP, mais embarquent des composants dont nous n'avons pas besoin dans ce projet (parsing complet, POS détaillé, etc.).

2.2.2 Modèle spaCy blank (coquille vide)

Dans notre cas, nous avons exploré :

- un modèle `spaCy blank` : un pipeline vide (`spacy.blank("fr")`) dans lequel nous ajoutons un composant NER entraîné de zéro sur notre dataset ;
- deux modèles basés sur les pipelines pré-entraînés `sm` et `md` : `fr_core_news_sm` et `fr_core_news_md`, sur lesquels le composant NER est affiné (*fine-tuning*) sur notre tâche spécifique.

Le modèle *blank* peut être vu comme une « coquille vide » : pas de connaissance linguistique préalable, tout est appris à partir de notre dataset spécialisé, ce qui évite d'embarquer des composants inutiles pour ce cas d'usage (POS, parsing générique, etc.).

2.3 Présentation de CamemBERT

CamemBERT est un modèle de langue français basé sur l'architecture RoBERTa, lui-même dérivé de BERT (Bidirectional Encoder Representations from Transformers). Quelques caractéristiques :

- architecture Transformer : empilement de blocs d'encodeurs avec mécanisme d'attention multi-têtes permettant de modéliser les dépendances longues dans le texte sans convolution ni récurrence ;
- pré-entraînement auto-supervisé sur un large corpus français (données OSCAR, plusieurs Go), via une tâche de *Masked Language Modeling* (prédire les tokens masqués) ;
- excellentes performances sur plusieurs tâches aval en français : POS tagging, parsing, NER, NLI.

2.3.1 Rappels sur BERT et les Transformers

BERT et ses variantes reposent sur une architecture Transformer bidirectionnelle, qui exploite des mécanismes d'attention pour modéliser le contexte à longue portée dans les phrases, sans recourir aux réseaux récurrents ni aux convolutions.

2.3.2 CamemBERT pour la NER en français

Dans notre projet, CamemBERT est utilisé comme encodeur de phrase dont la sortie alimente une couche de classification de tokens pour la NER (origine/destination). Nous avons entraîné plusieurs variantes (1k, 3k, 5k, full 7k) en ajustant uniquement quelques couches de sortie (*fine-tuning*) avec une loss de type *cross-entropy* sur les labels de tokens.

Contrairement à un réseau convolutionnel classique (CNN) qui regarde des fenêtres locales de mots, le Transformer utilise l'attention pour pondérer l'importance de chaque token par rapport aux autres, ce qui le rend très efficace pour capturer le contexte à longue portée dans les phrases.

Chapitre 3

Protocole expérimental

3.1 Dataset et splits (1000, 3000, 5000, full)

Le dataset complet est constitué d'environ 10 000 phrases, générées à partir de templates et de listes de villes/gares SNCF. Il est découpé ainsi :

- *train* : 7000 exemples ;
- *validation* : 1500 exemples ;
- *test* : 1500 exemples.

Un sous-dataset de 1000 exemples est extrait pour :

- expérimenter rapidement différentes configurations (notamment spaCy blank 1k et CamemBERT 1k) ;
- évaluer dans quelle mesure un petit volume de données bien ciblé peut suffire pour un MVP.

Les fichiers de données sont structurés en CSV avec, pour chaque phrase, les annotations BIO (ou équivalent) indiquant les tokens d'origine et de destination.

3.2 Paramètres d'entraînement (itérations, epochs, loss)

3.2.1 Entraînement spaCy

Les modèles spaCy sont entraînés via le script `train_spacy_models.py` avec les paramètres suivants :

- nombre d'itérations (`-n-iter`) : 30 ;
- bases testées : `blank`, `sm`, `md` ;
- tailles de dataset : 1000, 3000, 5000 et *full* (7000).

Les modèles produits sont :

- `ner_blank_1000`, `ner_blank_3000`, `ner_blank_5000`, `ner_blank_full` ;
- `ner_sm_full`, `ner_md_full`.

Chaque configuration est évaluée sur le jeu de test, avec les métriques suivantes :

- **Accuracy** (prédiction exacte des deux entités sur la phrase) ;
- **F1 Score global** sur les entités ;
- **Origin F1** et **Dest F1** (par type d'entité) ;
- temps d'inférence moyen par phrase.

3.2.2 Entraînement CamemBERT

Les modèles CamemBERT sont entraînés via `train_camembert.py` :

- tailles de train : 1000, 3000, 5000, 7000 ;
- nombre d'époques (`-epochs`) : 3 (avec différentes tailles de batch et learning rates, 16 et 5e-5 par défaut) ;
- métriques identiques à celles des modèles spaCy.

3.3 Post-traitement des entités (fonction `_clean_entity`)

Une attention particulière a été portée au post-processing des entités :

- initialement, les entités extraites contenaient parfois des espaces en début/fin ou de la ponctuation (par exemple « La Ferté-Imbault. ») ;
- une fonction `_clean_entity()` a été ajoutée pour nettoyer les entités (trim + suppression de la ponctuation finale).

Ce correctif a eu un impact massif sur les métriques :

- Accuracy d'environ 32 % à 80,33 % ;
- F1 global d'environ 78 % à 98,20 % sur le modèle `camembert_ner_1000`.

Cela illustre l'importance du post-processing dans un pipeline NER de production.

Chapitre 4

Résultats et analyses

4.1 Comparaison baseline / spaCy / CamemBERT

Tableau récapitulatif des principaux modèles sur le test :

Modèle	Accuracy	F1 Score	Origin F1	Dest F1	Inférence
CamemBERT 1k	80,33 %	98,20 %	98,14 %	98,26 %	124 ms
spaCy blank 1k	76,87 %	96,93 %	96,79 %	97,06 %	3,8 ms
Baseline (règles)	67,13 %	88,41 %	–	–	17,38 ms

Principaux constats :

- CamemBERT 1k obtient les meilleures performances en F1 et en accuracy, mais avec un temps d'inférence plus élevé (124 ms) et un modèle nettement plus lourd (ordre de centaines de Mo) ;
- spaCy blank 1k est environ 32 fois plus rapide (3,8 ms) pour une performance très proche ($F1 \approx 97\%$), avec un modèle léger (10 Mo) adapté à un déploiement à fort trafic ;
- la baseline reste nettement en dessous, ce qui justifie l'usage de modèles appris.

4.2 Impact de la taille du dataset

4.2.1 Comparaison des variantes spaCy

Modèle	Samples	Accuracy	F1 Score	Origin F1	Dest F1	Inférence
ner_blank_1000	1000	76,87 %	96,93 %	96,79 %	97,06 %	3,8 ms
ner_blank_3000	3000	76,73 %	96,77 %	96,70 %	96,83 %	3,8 ms
ner_blank_5000	5000	76,67 %	96,72 %	96,60 %	96,84 %	3,6 ms
ner_blank_full	7000	76,60 %	96,71 %	96,59 %	96,83 %	3,7 ms
ner_sm_full	7000	71,93 %	94,27 %	93,88 %	94,66 %	7,3 ms
ner_md_full	7000	72,00 %	94,27 %	93,85 %	94,69 %	9,5 ms

Constats :

- augmenter le nombre de samples de 1000 à 7000 n'apporte que des gains marginaux, voire une très légère baisse d'accuracy, probablement liée à la nature très homogène du dataset généré ;
- les modèles basés sur *sm* et *md* sont plus lents et légèrement moins bons que la version *blank 1k* sur cette tâche très spécialisée ;

- le modèle `ner_blank_1000` offre donc un compromis optimal : entraînement rapide, excellente F1, et inférence ultra-rapide.

Ce résultat confirme que, pour un cas d’usage restreint avec une structure de phrase relativement simple, un dataset de 1000 phrases annotées est « très suffisant » pour obtenir des performances quasi-SOTA, sans surconsommer les ressources.

4.2.2 Comparaison des variantes CamemBERT

Modèle	Samples	Accuracy	F1 Score	Origin F1	Dest F1	Inférence
<code>camembert_ner_1000</code>	1000	80,33 %	98,20 %	98,14 %	98,26 %	124 ms
<code>camembert_ner_3000</code>	3000	78,33 %	97,24 %	97,12 %	97,36 %	127 ms
<code>camembert_ner_5000</code>	5000	77,47 %	97,16 %	97,08 %	97,24 %	117 ms
<code>camembert_ner_full</code>	7000	75,93 %	96,66 %	96,54 %	96,79 %	133 ms

Constats :

- paradoxalement, `camembert_ner_1000` est le meilleur des quatre, suggérant qu’au-delà d’un certain point, ajouter plus de données synthétiques peut introduire du bruit ou conduire à un léger sur-apprentissage sur des patterns spécifiques ;
- les différences de F1 restent faibles, mais l’accuracy diminue légèrement avec la taille du dataset, ce qui peut être lié à des détails de post-processing ou de distribution de données ;
- le temps d’inférence reste relativement stable (120–130 ms), lié surtout à la taille fixe du modèle CamemBERT.

4.3 Sous- ou sur-apprentissage, courbes de loss

Les courbes de loss pendant l’entraînement (spaCy et CamemBERT) montrent généralement une convergence rapide en quelques époques/itérations, avec des pertes qui se stabilisent :

- sur 1000 samples, la perte baisse fortement au début, puis se stabilise sans remonter, ce qui suggère un entraînement suffisant sans sur-apprentissage massif ;
 - pour des jeux plus grands (3k, 5k, 7k), on observe une diminution plus progressive.
- Le cas d’un modèle à 35 % d’accuracy pour un F1 correct peut s’expliquer par :
- un mauvais post-processing des entités (décalages, ponctuation) qui fait échouer la correspondance exacte sur la phrase, alors que la détection de tokens reste bonne ;
 - un déséquilibre entre les classes ou une mauvaise gestion des séquences dans la fonction de loss.

C’est précisément ce qui a été observé avant l’introduction de `_clean_entity()` dans CamemBERT, avec un saut d’accuracy de 32 % à 80 % après correction.

4.4 Discussion : 1000 exemples comme compromis suffisant

Les résultats montrent qu’un sous-ensemble de 1000 phrases annotées permet déjà d’obtenir des performances très élevées avec des modèles spécialisés, tout en limitant le coût d’annotation et la complexité de l’entraînement.

Chapitre 5

Module de pathfinding

5.1 Modélisation du réseau de gares

Le réseau est modélisé comme un graphe pondéré :

- les nœuds représentent les gares (51 principales : Paris, Lyon, Marseille, Toulouse, Bordeaux, Lille, Nice, etc.) ;
- les arêtes représentent les liaisons ferroviaires (TGV, Intercités, TER) avec comme poids la durée de trajet.

La librairie NetworkX est utilisée pour manipuler ce graphe et exécuter l'algorithme de Dijkstra afin de trouver le plus court chemin en termes de temps. Exemples de routes :

- Paris → Marseille : 2 h 48, 666 km, 0 correspondance ;
- Lyon → Bordeaux : 3 h 22, 657 km, 2 correspondances ;
- Lille → Nice : 5 h 30, 1100 km, 1 correspondance.

5.2 Algorithme de Dijkstra avec NetworkX

Dijkstra est un algorithme classique de plus court chemin sur graphe pondéré à poids positifs. Dans notre implémentation :

- la durée (en minutes) sert de poids ;
- on utilise les fonctions `dijkstra_path` et `dijkstra_path_length` de NetworkX pour calculer le chemin et sa durée totale.

5.3 Mesure de la durée et routes optimales

C'est un excellent choix pour ce cas d'usage car :

- le graphe est de taille modérée (51 nœuds, 142 arêtes) ;
- on veut le plus court chemin selon un critère unique (temps) ;
- l'algorithme est déterministe, robuste, et très performant à cette échelle.

Une visualisation simple (graphe des gares avec la durée en minutes sur les arêtes) permet de montrer :

- la structure du réseau ;
- le plus court chemin entre deux gares en durée ;
- la comparaison intuitive entre plusieurs routes possibles (routes alternatives).

5.4 Intégration dans le pipeline complet

Pour l'expérimentation, on réutilise le même dataset de 1000 phrases pour tester la pipeline complète :

- NER (spaCy blank 1k ou CamemBERT 1k) pour extraire origine/destination ;
- Dijkstra sur le graphe de 51 gares pour calculer l'itinéraire.

Chapitre 6

MVP0 et déploiement

6.1 Choix du modèle final (spaCy blank 1k vs CamemBERT 1k)

Le MVP0 est une version minimale mais fonctionnelle du système, qui doit :

- prendre en entrée une phrase de requête en français ;
- extraire correctement l’origine et la destination dans la grande majorité des cas ;
- fournir un itinéraire cohérent avec durée, distance et correspondances ;
- répondre dans des temps acceptables (quelques millisecondes à quelques centaines de millisecondes).

Dans notre cas, un MVP0 répond à ces critères avec :

- `spaCy blank 1k` comme modèle par défaut (rapidité et précision suffisante) ;
- CamemBERT 1k comme option « haute précision » ;
- un graphe NetworkX des 51 gares ;
- une CLI ou une petite UI avec un simple champ de texte pour saisir la requête.

6.2 Considérations de performance et de ressources

Le choix entre spaCy et CamemBERT dépend du volume de requêtes, des contraintes de latence et des ressources disponibles (CPU/GPU, mémoire).

6.3 Interface utilisateur minimale (champ de texte)

Même une interface très simple est suffisante pour la démonstration :

- un champ de texte pour taper « Je veux aller de Paris à Marseille » ;
- un bouton « Résoudre » ;
- un affichage de la route calculée (liste des segments avec type de train, durée, distance) et de quelques métriques (durée totale, nombre de correspondances).

L’essentiel est de montrer la boucle complète : langage naturel → extraction des entités → calcul de chemin → réponse structurée.

6.4 Perspectives d'évolution

À terme, le système pourrait être étendu à d'autres types de transport, intégrer des données temps réel ou proposer des critères multiples d'optimisation (prix, émissions de CO₂, confort, etc.).

Chapitre 7

Conclusion

7.1 Bilan des expérimentations

Les expérimentations menées montrent la pertinence de combiner un modèle NER spécialisé avec un algorithme de plus court chemin pour résoudre des requêtes de voyage en langage naturel sur le réseau ferroviaire français.

Ce projet démontre qu'il est possible de passer d'une requête en langage naturel à un itinéraire de train concret en combinant NLP et pathfinding sur le réseau SNCF. Les expériences montrent que :

- CamemBERT 1k offre la meilleure précision (98.20 F1), au prix d'une latence plus élevée.
- SpaCy blank 1k fournit un excellent compromis (96.93 F1), 3.8 ms d'inférence.

La baseline à base de règles reste une solution simple et interprétable pour le prototypage. Selon les contraintes de précision, de latence et de coût d'infrastructure, le rapport recommande soit CamemBERT (applications critiques), soit spaCy (production à fort volume), avec la possibilité d'une architecture hybride pour combiner leurs forces.

7.2 Améliorations possibles

Parmi les pistes d'amélioration : enrichissement du dataset avec des requêtes plus variées, intégration d'autres langues, optimisation additionnelle des modèles ou déploiement sur une infrastructure scalable, collecter et annoter des données réelles d'utilisateurs.